Metal MLIR Dialect

Technical Documentation

A Custom MLIR Dialect for Apple Silicon GPU Compute

Pipeline: MLIR -> Metal Shading Language -> GPU Binary -> Apple Silicon

Target: Apple M-series GPUs via Metal API

Built against: LLVM/MLIR 22.1.2 (Homebrew)

Table of Contents

1. Project Overview

This project implements a custom MLIR dialect called 'metal' that targets Apple Silicon GPUs via Apple's Metal API. It provides an end-to-end compilation pipeline that takes GPU compute kernels written in MLIR, translates them to Metal Shading Language (MSL), compiles to GPU binary, and executes on the Apple GPU.

The key insight is that Apple's GPU ISA is not public - unlike NVIDIA's PTX or AMD's ISA. So instead of emitting raw GPU instructions, we emit MSL source code and let Apple's proprietary compiler (xcrun metal) handle the final compilation to GPU binary.

Why This Exists

MLIR has mature GPU backends for NVIDIA (nvvm/nvgpu -> PTX), AMD (rocdl/amdgpu -> AMDGCN), and Intel (gpu -> SPIR-V). Apple Metal has NO official MLIR dialect. This project fills that gap by providing a Metal dialect that maps Metal GPU concepts into MLIR's type system and operation framework.

What It Does

- Defines Metal-specific MLIR operations (kernel, thread_position_in_grid, barrier, etc.)
- Parses and validates Metal MLIR programs via 'metal-opt'
- Translates Metal MLIR to valid MSL source code via 'metal-translate'
- Compiles MSL to GPU binary (.metallib) using Apple's toolchain
- Executes GPU compute on Apple Silicon and verifies results

2. Architecture & Pipeline

The compilation pipeline has 5 stages:

End-to-End Pipeline

```
  Stage 1: MLIR (Metal dialect)
   |  metal.kernel, metal.thread_position_in_grid, memref.load/store, arith ops
   |  [metal-opt validates]
   v
Stage 2: Metal Shading Language (.metal)
   |  Valid MSL source code with kernel void, [[buffer(N)]], etc.
   |  [metal-translate --mlir-to-msl]
   v
Stage 3: Apple Intermediate Representation (.air)
   |  Apple's proprietary IR format
   |  [xcrun -sdk macosx metal -c]
   v
Stage 4: GPU Binary (.metallib)
   |  Loadable GPU binary
   |  [xcrun -sdk macosx metallib]
   v
Stage 5: Execution on Apple GPU
     Metal API loads .metallib, dispatches compute, reads results
     [runtime/runner.mm]
```

Stages 1-2 are what this project implements. Stages 3-4 use Apple's standard toolchain. Stage 5 uses Apple's Metal API via Objective-C++.

Metal Concept Mapping

Metal Concept	MLIR Op	MSL Output
kernel void	metal.kernel @name(...)	kernel void name(...)
thread_position_in_grid	metal.thread_position_in_grid x	_tid.x
threadgroup_barrier	metal.threadgroup_barrier 3	threadgroup_barrier(...)
device float* [[buffer]]	memref<?xf32> argument	device float* v [[buffer(N)]]
return (void)	metal.return	(implicit)
a + b	arith.addf %a, %b : f32	float v = v0 + v1;
buf[id]	memref.load %buf[%id]	v0[v1]

3. Directory Structure

Project Layout

```
 metal-dialect/
|-- CMakeLists.txt                    # Top-level build config
|-- include/
|   +-- MetalDialect/
|       |-- CMakeLists.txt            # TableGen targets
|       |-- MetalDialect.td           # Dialect registration
|       |-- MetalOps.td               # Operation definitions
|       +-- MetalDialect.h            # C++ header
|-- lib/
|   +-- MetalDialect/
|       |-- CMakeLists.txt            # Library build
|       |-- MetalDialect.cpp          # Dialect + KernelOp parser/printer
|       +-- TranslateToMSL.cpp        # MSL code emitter
|-- tools/
|   |-- metal-opt/
|   |   |-- CMakeLists.txt            # Tool build
|   |   +-- metal-opt.cpp             # MLIR optimizer/validator
|   +-- metal-translate/
|       |-- CMakeLists.txt            # Tool build
|       +-- metal-translate.cpp       # MLIR-to-MSL translator
|-- runtime/
|   |-- runner.mm                     # Obj-C++ GPU host program
|   |-- kernel.metal                  # Generated MSL source
|   |-- kernel.air                    # Compiled AIR
|   +-- kernel.metallib               # GPU binary
|-- test/
|   +-- vector_add.mlir               # Test kernel
+-- build/                            # CMake build output
```

What Each Layer Does

- **include/** include/ - Declarative definitions. TableGen (.td) files declare what ops exist, their arguments, results, and traits. The C++ header pulls in auto-generated code.
- **lib/** lib/ - Imperative implementation. The dialect initialization, custom parser/printer for KernelOp, and the MSL emitter that walks IR and outputs text.
- **tools/** tools/ - Executables. metal-opt validates IR, metal-translate converts IR to MSL.
- **runtime/** runtime/ - GPU execution. Objective-C++ program that loads .metallib and dispatches compute work to the GPU.
- **test/** test/ - MLIR input files for testing the pipeline.

4. Build System (CMakeLists.txt)

The project uses CMake with Ninja and links against Homebrew's pre-built LLVM/MLIR. This is an 'out-of-tree' build - the dialect lives outside the LLVM source tree.

Top-level CMakeLists.txt (annotated)

```
  cmake_minimum_required(VERSION 3.20)
project(metal-dialect LANGUAGES C CXX)
set(CMAKE_CXX_STANDARD 17)

find_package(MLIR REQUIRED CONFIG)    # Find MLIR CMake config
find_package(LLVM REQUIRED CONFIG)    # Find LLVM CMake config

# Import MLIR/LLVM CMake helpers
include(TableGen)                     # For mlir_tablegen()
include(AddLLVM)                      # For add_llvm_executable()
include(AddMLIR)                      # For add_mlir_library()

# Include paths for LLVM headers + our headers + generated headers
include_directories(${LLVM_INCLUDE_DIRS})
include_directories(${MLIR_INCLUDE_DIRS})
include_directories(${CMAKE_CURRENT_SOURCE_DIR}/include)
include_directories(${CMAKE_CURRENT_BINARY_DIR}/include)
```

How to Build

Build Commands

```
  cd metal-dialect

# Configure (point to Homebrew LLVM)
cmake -S . -B build -G Ninja \
  -DMLIR_DIR="$(brew --prefix llvm)/lib/cmake/mlir" \
  -DLLVM_DIR="$(brew --prefix llvm)/lib/cmake/llvm"

# Build
cmake --build build
```

The build produces two executables: build/tools/metal-opt/metal-opt and build/tools/metal-translate/metal-translate.

TableGen CMakeLists (include/MetalDialect/CMakeLists.txt)

This file runs mlir-tblgen to auto-generate C++ code from .td files:

TableGen Code Generation

```
  set(LLVM_TARGET_DEFINITIONS MetalOps.td)
mlir_tablegen(MetalOps.h.inc -gen-op-decls)       # Op class declarations
mlir_tablegen(MetalOps.cpp.inc -gen-op-defs)      # Op class definitions
mlir_tablegen(MetalDialect.h.inc -gen-dialect-decls) # Dialect class decl
mlir_tablegen(MetalDialect.cpp.inc -gen-dialect-defs) # Dialect class def
mlir_tablegen(MetalOpsEnums.h.inc -gen-enum-decls)   # Enum decls
mlir_tablegen(MetalOpsEnums.cpp.inc -gen-enum-defs)  # Enum defs
add_public_tablegen_target(MetalDialectIncGen)
```

These .inc files contain hundreds of lines of auto-generated C++ - parser/printer boilerplate, verifier logic, accessor methods, etc. You never edit them; you edit the .td files and regenerate.

5. Dialect Definition (TableGen)

TableGen is LLVM's domain-specific language for declaratively defining operations, types, and attributes. You describe WHAT your ops look like; the build system generates the C++ code that implements parsing, printing, verification, and accessors.

5.1 MetalDialect.td - Dialect Registration

MetalDialect.td

```
  include "mlir/IR/OpBase.td"
include "mlir/Interfaces/SideEffectInterfaces.td"

def Metal_Dialect : Dialect {
  let name = "metal";           // Ops will be: metal.kernel, metal.return, ...
  let cppNamespace = "::metal"; // C++ namespace for generated code
  let description = "Dialect for Apple Metal GPU compute";
}
```

This registers "metal" as a dialect name. Every op in this dialect will be prefixed with "metal." in MLIR syntax. The cppNamespace means all generated C++ classes live in the ::metal namespace.

5.2 MetalOps.td - Operation Definitions

This file defines all the operations in the Metal dialect. Let's walk through each one.

Dimension Enum

```
  def Metal_DimX : I32EnumAttrCase<"x", 0>;
def Metal_DimY : I32EnumAttrCase<"y", 1>;
def Metal_DimZ : I32EnumAttrCase<"z", 2>;

def Metal_Dimension : I32EnumAttr<"Dimension", "GPU dimension",
    [Metal_DimX, Metal_DimY, Metal_DimZ]> {
  let cppNamespace = "::metal";
}
```

GPU threads are organized in 3D grids. This enum represents which axis (x, y, z) a thread index op refers to. Maps directly to Metal's uint3 thread position.

KernelOp - GPU Entry Point

```
  def Metal_KernelOp : Metal_Op<"kernel", [IsolatedFromAbove]> {
  let summary = "Metal compute kernel function";
  let arguments = (ins
    StrAttr:$sym_name,                    // Kernel name (@vector_add)
    TypeAttrOf<FunctionType>:$function_type // (memref, memref) -> ()
  );
  let regions = (region AnyRegion:$body);   // The kernel body
  let hasCustomAssemblyFormat = 1;          // Custom parser/printer in C++
}
```

IsolatedFromAbove means the kernel body cannot reference SSA values defined outside it - just like a real GPU kernel

which runs independently. The sym_name becomes the function name in MSL. The function_type describes buffer argument types. hasCustomAssemblyFormat = 1 means we write our own parse/print methods in C++ (see MetalDialect.cpp) instead of using a declarative format string.

Thread Index Ops

```
  def Metal_ThreadPositionInGridOp : Metal_Op<
    "thread_position_in_grid", [Pure]> {
  let summary = "Get thread position in grid (global thread id)";
  let arguments = (ins Metal_Dimension:$dimension);
  let results = (outs Index:$result);
  let assemblyFormat = "$dimension attr-dict";
}
```

This is the most important op - it gives each GPU thread its unique ID. [Pure] means it has no side effects (it just reads a hardware register). It takes a dimension (x/y/z) and returns an Index value.

In Metal Shading Language, this maps to [[thread_position_in_grid]]. In CUDA terms, this is blockIdx.x * blockDim.x + threadIdx.x.

The three other thread ops (threadgroup_position_in_grid, thread_position_in_threadgroup, threads_per_threadgroup) follow the same pattern and map to Metal's [[threadgroup_position_in_grid]], [[thread_position_in_threadgroup]], and [[threads_per_threadgroup]] attributes respectively.

Barrier and Return Ops

```
  def Metal_ThreadgroupBarrierOp : Metal_Op<"threadgroup_barrier", []> {
  let arguments = (ins I32Attr:$mem_flags);  // 0=none, 1=device, 2=threadgroup, 3=both
  let assemblyFormat = "$mem_flags attr-dict";
}

def Metal_ReturnOp : Metal_Op<"return", [Terminator]> {
  let assemblyFormat = "attr-dict";
}
```

The barrier synchronizes threads within a threadgroup - all threads must reach the barrier before any can proceed. [Terminator] on ReturnOp tells MLIR this op ends a block.

6. C++ Implementation

6.1 MetalDialect.h - The Header

MetalDialect.h

```
 #pragma once
#include "mlir/IR/Dialect.h"
#include "mlir/IR/OpDefinition.h"
#include "mlir/IR/Builders.h"
#include "mlir/IR/BuiltinTypes.h"
#include "mlir/Bytecode/BytecodeOpInterface.h"
#include "mlir/Interfaces/SideEffectInterfaces.h"
#include "mlir/Interfaces/InferTypeOpInterface.h"

#include "MetalDialect/MetalDialect.h.inc"    // Generated dialect class
#include "MetalDialect/MetalOpsEnums.h.inc"    // Generated Dimension enum

#define GET_OP_CLASSES
#include "MetalDialect/MetalOps.h.inc"         // Generated op classes
```

This header does very little manually - it just includes the auto-generated .h.inc files that TableGen produced. The #define GET_OP_CLASSES before MetalOps.h.inc tells the generated code to emit actual class definitions (KernelOp, ThreadPositionInGridOp, etc.).

6.2 MetalDialect.cpp - Dialect Initialization + KernelOp Parser

This file has two responsibilities: initializing the dialect (registering all ops) and providing the custom parse/print methods for KernelOp.

Dialect Initialization

```
 void metal::MetalDialect::initialize() {
  addOperations<
#define GET_OP_LIST
#include "MetalDialect/MetalOps.cpp.inc"   // Expands to: KernelOp, ThreadPosi...
  >();
}
```

GET_OP_LIST expands to a comma-separated list of all op classes. addOperations() registers them with the MLIR context so the parser recognizes metal.* ops.

KernelOp Parser (simplified)

```
  // How MLIR text like this gets parsed:
//   metal.kernel @vector_add(%a: memref<?xf32>, %b: memref<?xf32>) {
//     ...
//   }

ParseResult metal::KernelOp::parse(OpAsmParser &parser, OperationState &result) {
  // 1. Parse kernel name: @vector_add
  StringAttr nameAttr;
  parser.parseSymbolName(nameAttr, "sym_name", result.attributes);

  // 2. Parse arguments: (%a: memref<?xf32>, %b: memref<?xf32>)
  SmallVector<OpAsmParser::Argument> args;
  SmallVector<Type> argTypes;
  parser.parseLParen();
  do {
    // Parse each: %name : type
    OpAsmParser::Argument arg;
    Type argType;
    parser.parseArgument(arg);
    parser.parseColon();
    parser.parseType(argType);
    arg.type = argType;
    args.push_back(arg);
  } while (succeeded(parser.parseOptionalComma()));
  parser.parseRParen();

  // 3. Build function type and parse body region
  auto funcType = FunctionType::get(parser.getContext(), argTypes, {});
  result.addAttribute("function_type", TypeAttr::get(funcType));
  auto *body = result.addRegion();
  parser.parseRegion(*body, args);
}
```

7. MSL Translation (TranslateToMSL.cpp)

This is the core of the code generation. The MSLEmitter class walks the MLIR IR tree and outputs valid Metal Shading Language source code. It's registered as a translation pass that can be invoked with --mlir-to-msl.

How It Works

The emitter uses a simple pattern: visit each MLIR operation, check its type via dyn_cast<>, and emit the corresponding MSL text. SSA values (%0, %1, ...) are mapped to C variable names (v0, v1, ...) via a DenseMap.

Variable Name Tracking

```
 class MSLEmitter {
  raw_ostream &os;                          // Output stream
  DenseMap<Value, std::string> valueNames;  // %0 -> "v0", %1 -> "v1"
  int nextVar = 0;                           // Counter for variable names

  StringRef getName(Value v) {
    // Lazily assign a name: first time we see a value, assign "vN"
    auto it = valueNames.find(v);
    if (it != valueNames.end()) return it->second;
    std::string name = "v" + std::to_string(nextVar++);
    valueNames[v] = name;
    return valueNames[v];
  }
};
```

Kernel Emission

When the emitter encounters a metal.kernel op, it:

- Emits 'kernel void <name>(' as the function signature
- For each memref argument, emits 'device float* vN [[buffer(N)]]'
- Adds 'uint _tid [[thread_position_in_grid]]' as the last parameter
- Walks all ops in the kernel body and emits each one

Op-to-MSL Mapping

Operation Dispatch (simplified)

```
  void emitOp(Operation &op) {
  if (auto threadId = dyn_cast<metal::ThreadPositionInGridOp>(op))
    // metal.thread_position_in_grid x  ->  uint v0 = _tid;
    os << "    uint " << getName(op) << " = _tid;\n";

  else if (auto load = dyn_cast<memref::LoadOp>(op))
    // memref.load %buf[%id]  ->  float v1 = v0[v2];
    os << "    float " << getName(result) << " = " << getName(buf)
       << "[" << getName(idx) << "];\n";

  else if (auto addf = dyn_cast<arith::AddFOp>(op))
    // arith.addf %a, %b  ->  float v3 = v1 + v2;
    os << "    float " << getName(result) << " = " << getName(lhs)
       << " + " << getName(rhs) << ";\n";

  else if (auto store = dyn_cast<memref::StoreOp>(op))
    // memref.store %val, %buf[%id]  ->  v0[v2] = v3;
    os << "    " << getName(buf) << "[" << getName(idx) << "] = "
       << getName(val) << ";\n";
}
```

Registration

Translation Registration

```
  void registerToMSLTranslation() {
  TranslateFromMLIRRegistration reg(
    "mlir-to-msl",       // CLI flag: --mlir-to-msl
    "Translate Metal MLIR to Metal Shading Language",
    [](ModuleOp module, raw_ostream &os) {
      MSLEmitter emitter(os);
      emitter.emitPreamble();            // #include <metal_stdlib>
      module.walk([&](metal::KernelOp kernel) {
        emitter.emitKernel(kernel);       // Emit each kernel
      });
      return success();
    },
    [](DialectRegistry &registry) { ... } // Register needed dialects
  );
}
```

8. Tools

8.1 metal-opt - IR Optimizer/Validator

metal-opt.cpp

```
 int main(int argc, char **argv) {
  mlir::DialectRegistry registry;
  registry.insert<metal::MetalDialect>();    // Our dialect
  registry.insert<mlir::arith::ArithDialect>(); // For arith.addf, etc.
  registry.insert<mlir::memref::MemRefDialect>(); // For memref.load/store
  registry.insert<mlir::func::FuncDialect>();     // For func.func

  return mlir::asMainReturnCode(
    mlir::MlirOptMain(argc, argv, "Metal MLIR optimizer\n", registry));
}
```

metal-opt is a customized version of mlir-opt that knows about the Metal dialect. It can parse, validate, and print Metal IR. It also supports all built-in MLIR passes (--canonicalize, --cse, etc.) for any standard dialect ops used inside kernels.

Usage: ./build/tools/metal-opt/metal-opt test/vector_add.mlir

8.2 metal-translate - MSL Code Generator

metal-translate.cpp

```
 int main(int argc, char **argv) {
  registerToMSLTranslation();  // Register --mlir-to-msl
  return failed(
    mlir::mlirTranslateMain(argc, argv, "Metal MLIR translator\n"));
}
```

metal-translate converts Metal MLIR into MSL source code. The registerToMSLTranslation() call (from TranslateToMSL.cpp) makes the --mlir-to-msl flag available.

Usage: ./build/tools/metal-translate/metal-translate --mlir-to-msl test/vector_add.mlir -o kernel.metal

9. GPU Runtime (runner.mm)

The runtime is an Objective-C++ program that uses Apple's Metal API to load the compiled GPU binary and execute the kernel. It's the 'host' side of GPU compute.

Execution Steps

- **Step 1:** Get GPU device via MTLCreateSystemDefaultDevice()
- **Step 2:** Load .metallib file via [device newLibraryWithURL:]
- **Step 3:** Look up kernel function by name via [library newFunctionWithName:]
- **Step 4:** Create compute pipeline via [device newComputePipelineStateWithFunction:]
- **Step 5:** Allocate input data on CPU (two float arrays: [0..1023] and [0,2,4..2046])
- **Step 6:** Create Metal buffers with MTLResourceStorageModeShared (unified memory on Apple Silicon - no CPU<->GPU copy needed!)
- **Step 7:** Create command queue, command buffer, and compute encoder
- **Step 8:** Bind pipeline and buffers to encoder at indices 0, 1, 2 (matching [[buffer(N)]])
- **Step 9:** Dispatch 1024 threads in a 1D grid
- **Step 10:** Commit and wait for GPU completion
- **Step 11:** Read results directly from shared buffer and verify a[i] + b[i] == result[i]

Key Detail: Unified Memory

On Apple Silicon, CPU and GPU share the same physical memory. Using MTLResourceStorageModeShared means the Metal buffer IS the CPU buffer - no copying. After the GPU finishes, you read results directly with [bufOut contents]. This is a major advantage over discrete GPUs (NVIDIA/AMD) where you must explicitly copy data between CPU and GPU memory.

Compile & Run

Runtime Build

```
  # Compile the Objective-C++ runner
clang++ -framework Metal -framework Foundation \
        -std=c++17 -ObjC++ runner.mm -o runner

# Run with a metallib
./runner kernel.metallib
```

10. Test Kernel (vector_add.mlir)

test/vector_add.mlir (annotated)

```
 module {
 metal.kernel @vector_add(
     %a: memref<?xf32>,     // Input buffer A (device float*)
     %b: memref<?xf32>,     // Input buffer B (device float*)
     %out: memref<?xf32>    // Output buffer  (device float*)
 ) {
   // Get this thread's global index
   %id = metal.thread_position_in_grid x

   // Load a[id] and b[id]
   %va = memref.load %a[%id] : memref<?xf32>
   %vb = memref.load %b[%id] : memref<?xf32>

   // Compute sum
   %sum = arith.addf %va, %vb : f32

   // Store result[id] = sum
   memref.store %sum, %out[%id] : memref<?xf32>

   metal.return
 }
}
```

This translates to the following MSL:

Generated MSL Output

```
 #include <metal_stdlib>
using namespace metal;

kernel void vector_add(
    device float* v0 [[buffer(0)]],
    device float* v1 [[buffer(1)]],
    device float* v2 [[buffer(2)]],
    uint _tid [[thread_position_in_grid]]
) {
    uint v3 = _tid;
    float v4 = v0[v3];
    float v5 = v1[v3];
    float v6 = v4 + v5;
    v2[v3] = v6;
}
```

11. End-to-End Pipeline Walkthrough

Full Pipeline Commands

```
  # Step 1: Validate the Metal MLIR
./build/tools/metal-opt/metal-opt test/vector_add.mlir

# Step 2: Translate to MSL source code
./build/tools/metal-translate/metal-translate \
    --mlir-to-msl test/vector_add.mlir -o runtime/kernel.metal

# Step 3: Compile MSL -> AIR (Apple Intermediate Representation)
xcrun -sdk macosx metal -c runtime/kernel.metal -o runtime/kernel.air

# Step 4: Link AIR -> metallib (GPU binary)
xcrun -sdk macosx metallib runtime/kernel.air -o runtime/kernel.metallib

# Step 5: Execute on GPU
./runtime/runner runtime/kernel.metallib
```

Expected output:

GPU Output

```
  Using device: Apple M4

Results (first 10):
  0 + 0 = 0
  1 + 2 = 3
  2 + 4 = 6
  3 + 6 = 9
  4 + 8 = 12
  5 + 10 = 15
  6 + 12 = 18
  7 + 14 = 21
  8 + 16 = 24
  9 + 18 = 27

Verification: PASSED (0 errors out of 1024)
```

The runtime creates two arrays: A = [0, 1, 2, ..., 1023] and B = [0, 2, 4, ..., 2046]. The kernel computes A[i] + B[i] for all 1024 elements in parallel on the GPU. Each GPU thread handles one element.

12. How to Extend

Adding a New Op

To add a new operation (e.g. metal.grid_size):

- Add the op definition to MetalOps.td with arguments, results, and format
- Rebuild (cmake --build build) - TableGen auto-generates C++ code
- Add MSL emission for the op in TranslateToMSL.cpp's emitOp() method
- Write a test .mlir file that uses the new op

Adding New Kernel Types

The current emitter supports 1D kernels with float buffers. To extend:

- 2D/3D grids: emit uint3 _tid and use _tid.x, _tid.y, _tid.z in emitThreadId()
- Integer buffers: update emitKernel() to handle memref<?xi32> -> device int*
- Shared memory: add threadgroup address space support for threadgroup float* in MSL
- Loops: handle scf.for by emitting MSL for-loops
- Conditionals: handle scf.if by emitting MSL if/else

Adding a Lowering Pass (gpu -> metal)

To lower from MLIR's generic gpu dialect to this Metal dialect, write a ConversionPass that rewrites gpu.thread_id -> metal.thread_position_in_grid, gpu.barrier -> metal.threadgroup_barrier, etc. Register it in metal-opt with PassRegistration<>.

Data Flow Diagram

Build & Data Flow

```
.td files          mlir-tblgen        .h.inc / .cpp.inc
(you write)   -->  (auto-generates)   (you never edit)
     |                                       |
     v                                       v
MetalDialect.cpp  <---  includes  <---  MetalDialect.h
(you write)                              (you write)
     |
     v
libMetalDialect.a  -->  metal-opt (validate IR)
     |              -->  metal-translate (emit MSL)
     v
kernel.metal  -->  xcrun metal  -->  kernel.metallib  -->  runner (GPU)
```